Caesar, Playfair and Hill

How each algorithm works between Entity A and Entity B

1. Caesar

What A and B share beforehand

A and B agree on two things in advance. Together they are the secret key:

- **the shift n**, a whole number such as 3,
- **the direction**: “Gauche → Droite” moves forward in the alphabet (+n), “Droite → Gauche” moves backward (−n).

In the interface, these are the “Décalage n” and “Sens” fields.

The alphabet used

In classic Caesar only the 26 capital letters are shifted. In this TP the alphabet is extended to fix that weakness. It is defined in `chiffrement.py`:

```
ABC…Z  abc…z  0123456789  (space)      →  63 symbols
index: 0…25   26…51   52…61       62
```

Any character outside this list stays unchanged, for example `,` `-` é.

Entity A encrypts

For each character at position i in the alphabet:

```
encrypted = ALPHABET[(i + n) mod 63]     (Gauche → Droite)
encrypted = ALPHABET[(i − n) mod 63]     (Droite → Gauche)
```

The “mod 63” makes the alphabet wrap around, so after 9 and space it starts again at A.

Example (run with the project code): message `Salut B, rdv 9h`, n = 3, Gauche → Droite.

	S	**a**	**l**	**u**	**t**	**␣**	**B**	**,**	**␣**	**r**	**d**	**v**	**␣**	**9**	**h**
Index	18	26	37	46	45	62	1	–	62	43	29	47	62	61	33
+3 mod 63	21	29	40	49	48	2	4	–	2	46	32	50	2	1	36
Encrypted	V	d	o	x	w	C	E	,	C	u	g	y	C	B	k

A sends `VdoxwCE,CugyCBk` to B. The spaces are encrypted too: they became C. That hides where the words start and end.

Entity B decrypts

B receives `VdoxwCE,CugyCBk` and applies the **opposite shift** with the same key:

```
clear = ALPHABET[(i − n) mod 63]
```

For example, V (21) − 3 = 18 → S, and C (2) − 3 = −1 mod 63 = 62 → space.

B gets back exactly `Salut B, rdv 9h`. In the code, `cesar_dechiffrer` simply calls `cesar_chiffrer` with the opposite direction. If A had chosen Droite → Gauche, the same message would give `PXirq79,7oas76e`, and B would decrypt by shifting forward instead.

The weaknesses handled in the TP

Weakness of classic Caesar	What the code does
Only 25 possible keys, so brute force is instant	Extended alphabet: 62 useful keys. Still small, but better.
Spaces and capitals stay visible, so the word structure leaks	Spaces, lowercase letters and digits are encrypted too
A shift that is a multiple of the alphabet size (0, 63, 126…) gives back the clear text	Refused with the error “Clé faible : un décalage multiple de 63 ne chiffre pas le message”

Limit to mention in the report: even with 63 symbols there are only 62 useful keys, so brute force still breaks it in a fraction of a second. Frequency analysis also still works, because the same letter is always encrypted the same way. Caesar shows the principle of symmetric encryption, but it is not secure.

2. Playfair

What A and B share beforehand

A and B agree on a **secret keyword**, for example SECURITE. In the interface, this is the “Mot clé” field.

Building the 6×6 matrix (both sides do it)

From the keyword, A and B each build the **same grid**:

- Write the letters of the keyword, skipping repeats: S E C U R I T (the second E is dropped).
- Fill the rest with the unused symbols, in order: A…Z then 0…9.

Classic Playfair uses a 5×5 grid with only letters, where I and J are merged. In this TP the grid is **6×6 with 36 symbols**, letters and digits. Grid built for SECURITE:

S	E	C	U	R	I
T	A	B	D	F	G
H	J	K	L	M	N
O	P	Q	V	W	X
Y	Z	0	1	2	3
4	5	6	7	8	9

Entity A prepares the message

Example message: `Ballon 2026`

- **Clean it:** convert to capitals and keep only letters and digits. That gives BALLON2026; the space is dropped.
- **Cut it into pairs.** If a pair would contain the same letter twice, a padding letter is put between them. If one letter is left alone at the end, a padding letter is added. The padding letter is X, or Q if the letter itself is X.

```
BA  LX  LO  N2  02  6X
```

Entity A encrypts each pair (3 rules)

Rule	Encryption
Same row	each symbol is replaced by the one **to its right**; the row wraps around
Same column	each symbol is replaced by the one **below it**; the column wraps around
Rectangle (different row and column)	each symbol keeps **its row** and takes **the other symbol’s column**

Pair	Positions (row, col)	Rule	Result
BA	B(1,2) A(1,1)	same row → shift right	**DB**
LX	L(2,3) X(3,5)	rectangle	**NV**
LO	L(2,3) O(3,0)	rectangle	**HV**
N2	N(2,5) 2(4,4)	rectangle	**M3**
02	0(4,2) 2(4,4)	same row → shift right	**13**
6X	6(5,2) X(3,5)	rectangle	**9Q**

A sends `DBNVHVM3139Q` to B. This example has no “same column” pair. One would work like this: EA, with E (0,1) above A (1,1), becomes AJ.

Entity B decrypts

B builds the same grid from SECURITE, cuts the received text into pairs and applies the **reverse rules**: same row → shift **left**, same column → shift **up**, rectangle → same rule as for encryption (it is its own reverse).

```
DB NV HV M3 13 9Q  →  BA LX LO N2 02 6X  →  BALXLON2026X
```

Finally, B **removes the padding letters** (function `playfair_enlever_bourrage`): the X between the two L’s is removed, and so is the X at the very end. B gets back `BALLON2026`, which is exactly what the code produced when run.

The weaknesses handled in the TP

Weakness of classic Playfair	What the code does
I and J are merged, so B can’t know which one was meant	6×6 grid: every letter has its own cell
Digits are lost	Digits 0–9 are in the grid
Padding X letters stay in the decrypted message	Padding is removed automatically after decryption
X followed by X can’t be padded with X	Padding letter is Q when the letter is X
An empty key gives a grid that is just the plain alphabet	Empty key refused

Limits to mention in the report

- **Lowercase letters, spaces and punctuation are lost.** B receives BALLON2026, not “Ballon 2026”.
- **Padding removal can be ambiguous.** A message that really contains an X between two identical letters will lose that X. For example, AXA comes back as AA.
- **It is still breakable.** Playfair encrypts pairs rather than single letters, so simple letter-frequency analysis no longer works. But the same pair always gives the same result, so a pair-frequency analysis on a long text can still break it.

3. Hill

What A and B share beforehand

A and B agree on a **secret key matrix** of numbers, 2×2 or 3×3 (the “Matrice” cells in the interface). Example, the default key:

```
K = | 3  3 |
    | 2  5 |
```

The alphabet is the 26 letters, each turned into a number:

```
A=0  B=1  C=2  D=3  E=4  F=5  G=6  H=7  I=8  J=9  K=10 L=11 M=12
N=13 O=14 P=15 Q=16 R=17 S=18 T=19 U=20 V=21 W=22 X=23 Y=24 Z=25
```

Checking the key (both sides)

B must be able to **reverse** the matrix, otherwise decryption is impossible. The code (`hill_cle`) checks this before doing anything:

```
det(K) = 3×5 − 3×2 = 9
gcd(9, 26) = 1   →  9 has an inverse mod 26  →  valid key
```

Entity A encrypts

Example message: `Hello`

- **Clean it:** capitals only, letters only. That gives HELLO.
- **Cut it into blocks** of the matrix size (2 here). If the last block is incomplete, add X. That gives HE LL OX.
- **Multiply each block by the matrix**, then take the result mod 26:

```
| c1 |   | 3  3 |   | p1 |
| c2 | = | 2  5 | × | p2 |   mod 26
```

Block	Numbers	Calculation	Result
HE	(7, 4)	3·7+3·4 = 33 → **7** ; 2·7+5·4 = 34 → **8**	**HI**
LL	(11, 11)	3·11+3·11 = 66 → **14** ; 2·11+5·11 = 77 → **25**	**OZ**
OX	(14, 23)	3·14+3·23 = 111 → **7** ; 2·14+5·23 = 143 → **13**	**HN**

A sends `HIOZHN` to B. This shows Hill’s main strength: the two L’s became O and Z, two different letters. The same letter does not always give the same result, because each letter depends on its neighbour in the block.

Entity B decrypts

B computes the **inverse matrix mod 26** (function `hill_inverse`):

```
det = 9      inverse of 9 mod 26 = 3      (because 9×3 = 27 ≡ 1)

K⁻¹ = 3 × |  5  −3 |  mod 26  =  | 15  17 |
          | −2   3 |             | 20   9 |
```

B then multiplies each received block by K^{-1}:

Received	Numbers	Calculation	Result
HI	(7, 8)	15·7+17·8 = 241 → **7** ; 20·7+9·8 = 212 → **4**	**HE**
OZ	(14, 25)	15·14+17·25 = 635 → **11** ; 20·14+9·25 = 505 → **11**	**LL**
HN	(7, 13)	15·7+17·13 = 326 → **14** ; 20·7+9·13 = 257 → **23**	**OX**

B gets back HELL0X, which is exactly what the code produced when run. A 3×3 matrix works the same way, with blocks of 3 letters; its inverse is computed with cofactors, which the code also handles.

The weaknesses handled in the TP

Weakness of Hill	What the code does
A matrix whose determinant has no inverse mod 26 makes decryption impossible	Refused. For example, “2 4 1 2” gives “le déterminant (0) n’est pas inversible modulo 26”
The identity matrix “1 0 0 1” leaves the message unchanged	Refused: “Clé faible : la matrice identité ne chiffre pas le message”
A small 2×2 key is easy to break	3×3 matrix allowed: more possible keys, and each letter depends on its 2 neighbours
The ciphertext length is not a multiple of the block size	Refused on decryption

Limits to mention in the report

- **The padding X stays.** B gets HELLOX, not HELLO. Unlike Playfair, the code does not remove the padding after decryption.
- **Spaces, digits, lowercase letters and punctuation are lost.**
- **Known-plaintext attack.** Hill is linear. If an attacker knows a few clear/encrypted pairs (2 blocks for 2×2, 3 blocks for 3×3), they can solve the equations and find the key directly. The 3×3 option makes this harder, but does not prevent it.